



MOBILE APPLICATION DEVELOPMENT

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS



AULA 01

Construindo UIs com dados remotos

Lidando com o ciclo de uma consulta



“Um sistema distribuído é aquele em que a falha de um computador que você nem sabia que existia pode tornar o seu próprio computador inutilizável.”

— Leslie Lamport



SITUAÇÃO-PROBLEMA

Um aplicativo precisa exibir uma lista de dados que está disponível em uma API.

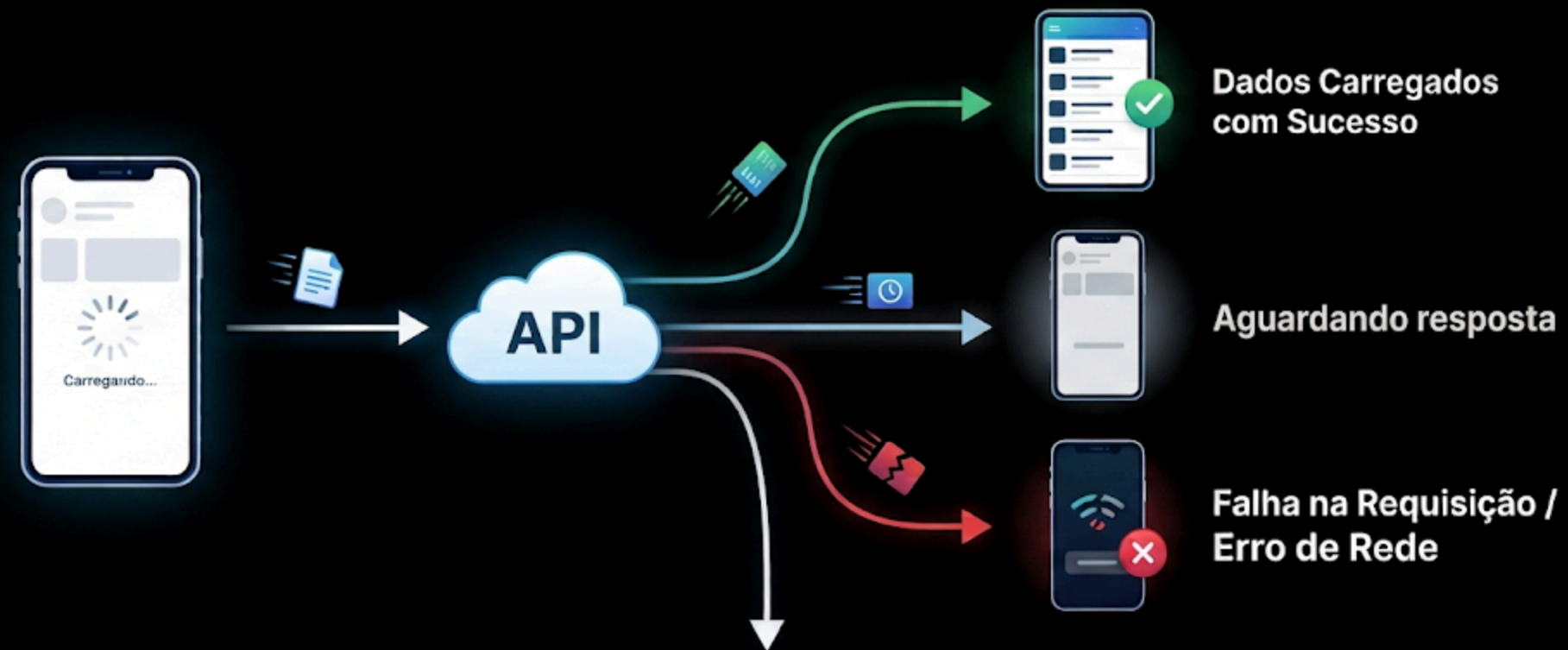
Quando o usuário abre a tela, porém, esses dados **ainda não estão no dispositivo**.

A aplicação precisa buscá-los e continuar oferecendo uma interface adequada enquanto essa consulta acontece.

Mas algumas coisas podem acontecer:

- a resposta pode demorar;
- os dados podem chegar corretamente;
- a requisição pode falhar.

COMO CONSTRUIR UMA INTERFACE QUE LIDE COM ESSAS SITUAÇÕES?



**Como construir uma tela
que lide com essas situações?**

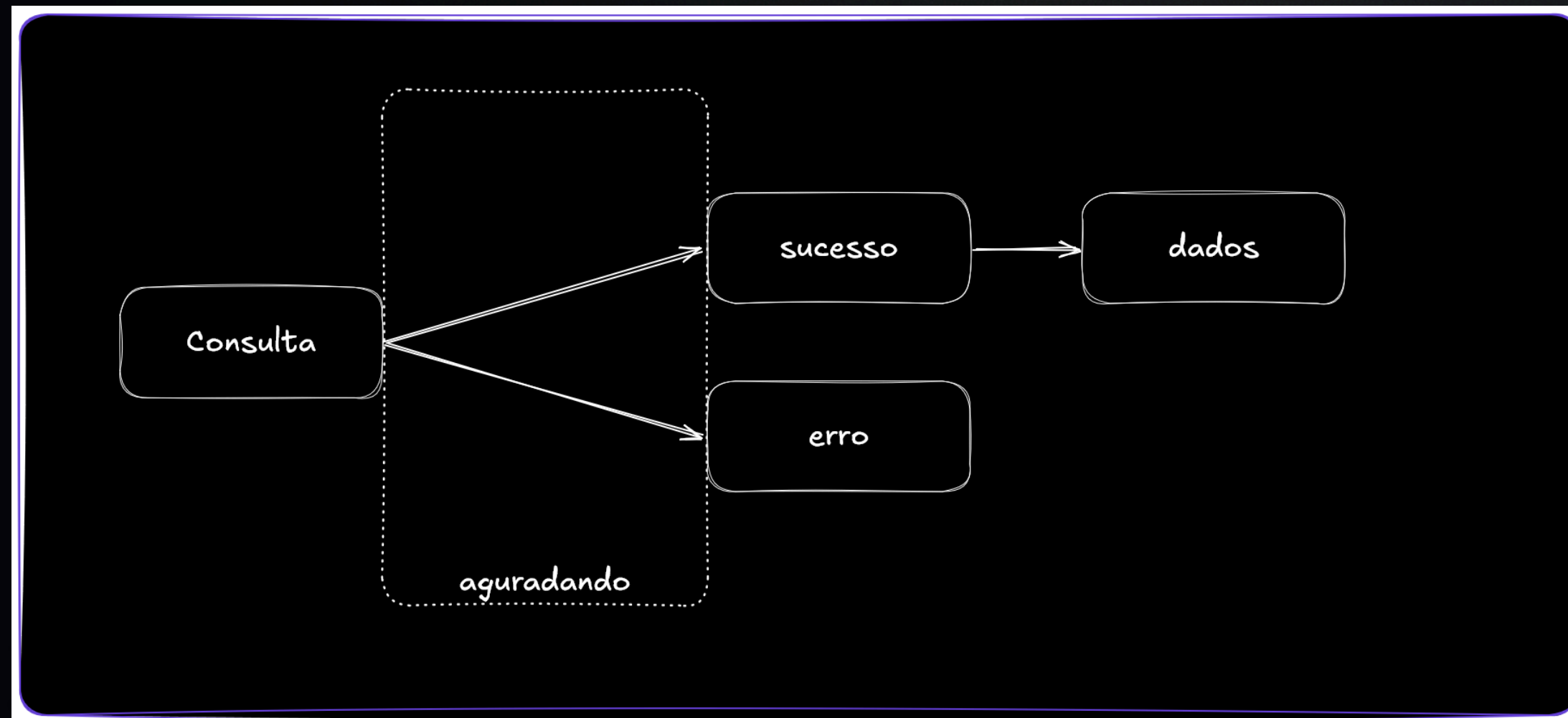
A SOLUÇÃO QUE JÁ CONHECEMOS

```
const [isLoading, setIsLoading] = useState(false);
const [error, setError] = useState(null);

useEffect(() => {
  async function loadData() {
    try {
      setIsLoading(true);
      // recupera dados de uma fonte externa
    } catch (e) {
      setError(e.message);
    } finally {
      setIsLoading(false);
    }
  }

  await loadData();
}, []);
```


O CICLO DE UMA CONSULTA



Quando buscamos dados remotos, não gerenciamos apenas os dados. Precisamos gerenciar o estado da consulta.

**SE ESSE CICLO APARECE SEMPRE QUE
BUSCAMOS DADOS REMOTOS,
PRECISAMOS CONTROLAR TUDO ISSO
MANUALMENTE TODA VEZ?**

TANSTACK QUERY

Até aqui, controlamos manualmente o ciclo de uma consulta:

- quando ela está aguardando uma resposta;
- quando os dados chegam;
- quando ocorre um erro.

O **TanStack Query** é uma biblioteca que nos ajuda a gerenciar esse ciclo.

Ele **não substitui o `fetch`**.

Ele organiza o estado produzido pelas consultas de dados remotos.

UMA CONSULTA, UMA QUERY

No TanStack Query, uma consulta de leitura é chamada de **Query**.

Uma Query precisa responder basicamente a duas perguntas:

- **Qual consulta estamos fazendo?**
- **Como os dados serão buscados?**

```
queryKey → identifica a consulta  
queryFn  → busca os dados
```

A partir disso, o TanStack Query acompanha o ciclo da consulta para nós.

INSTALAÇÃO

Para usar o TanStack Query no projeto:

```
npm i @tanstack/react-query
```

Nesta aula, esse é o único pacote que precisamos instalar.

CONFIGURAÇÃO

Antes de criar nossas queries, precisamos disponibilizar o TanStack Query para a aplicação.

```
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";

const queryClient = new QueryClient();

function Layout() {
  return (
    <QueryClientProvider client={queryClient}>
      <Stack />
    </QueryClientProvider>
  );
}
```

O `QueryClientProvider` permite que os componentes da aplicação utilizem as queries gerenciadas pelo `queryClient`.

Em um projeto com Expo Router, normalmente fazemos essa configuração em:

```
app/_layout.tsx
```


USEQUERY

O `useQuery` representa uma **consulta de dados remotos**.

Para criar uma query, precisamos inicialmente de duas informações:

```
useQuery({  
  queryKey: ["movies"],  
  queryFn: fetchMovies,  
});
```

QUERYKEY

Identifica a consulta.

```
queryKey: ["movies"];
```

QUERYFN

Define como os dados serão buscados.

```
queryFn: fetchMovies;
```




NOSSA PRIMEIRA QUERY

```
const { data, isPending, error } = useQuery({
  queryKey: ["movies"],
  queryFn: fetchMovies,
});

if (isPending) {
  return <ActivityIndicator />;
}

if (error) {
  return <Text>Erro ao carregar: {error.message}</Text>;
}

return (
  <FlatList
    data={data}
    renderItem={({ item }) => <Text>{item.title}</Text>}
    keyExtractor={(item) => item.id.toString()}
  />
);
```




O CICLO DA CONSULTA

Lembra do problema que tínhamos no início da aula?

O `useQuery` nos informa em que situação a consulta se encontra:

Aguardando resposta → `isPending`

Dados recebidos → `data`

Falha na consulta → `error`

Agora a interface pode reagir a cada uma dessas situações.

ANTES E DEPOIS

ANTES

Precisávamos controlar manualmente:

```
const [data, setData] = useState([]);
const [isLoading, setIsLoading] = useState(false);
const [error, setError] = useState(null);

useEffect(() => {
  // buscar dados
  // alterar loading
  // tratar erro
  // atualizar dados
}, []);
```

COM UMA QUERY

```
const { data, isPending, error } = useQuery({
  queryKey: ["movies"],
  queryFn: fetchMovies,
});
```


A RESPONSABILIDADE DE ACOMPANHAR O
CICLO DA CONSULTA DEIXA DE FICAR
ESPALHADA PELO COMPONENTE.



AGORA É COM VOCÊ

A consulta foi realizada com sucesso.

Mas e se a API retornar uma lista vazia?

DESAFIO

Altere a tela para que:

- existindo tweets, seja exibida a quantidade;
- não existindo tweets, seja exibida a mensagem:

Nenhum tweet encontrado.

RECOMENDAÇÃO DE LEITURA

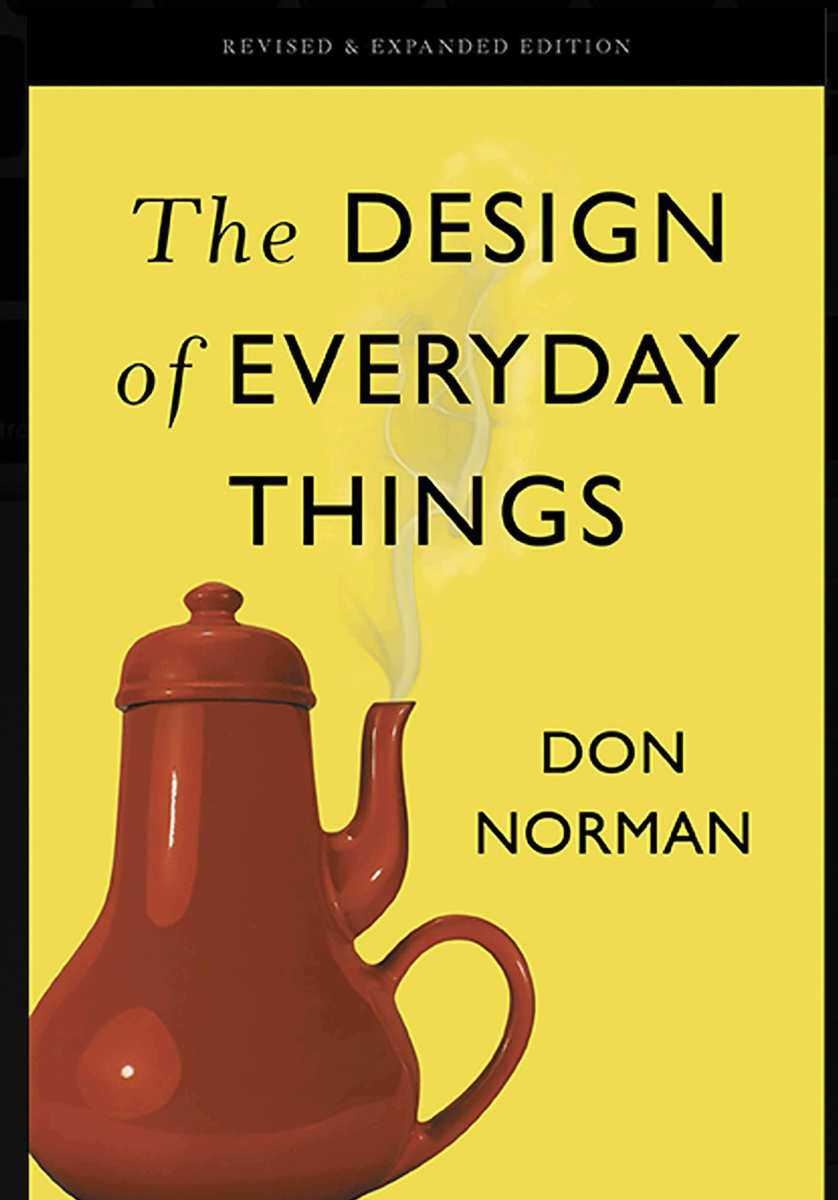
THE DESIGN OF EVERYDAY THINGS

Don Norman

Por que algumas coisas parecem naturais de usar enquanto outras nos deixam sem saber o que está acontecendo?

Don Norman explora princípios como **feedback**, visibilidade e a relação entre as ações do usuário e as respostas de um sistema.

Uma boa interface não deve apenas funcionar.
Ela deve comunicar o que está acontecendo.



LICENÇA

Este material é disponibilizado sob a licença Creative Commons Atribuição 4.0 Internacional (CC BY 4.0).

 Atribuição 4.0 Internacional

Autor: Felipe Cavalcante Vilella de Oliveira, 2026